

Guía práctica de SQLJ con Java 8

El contenido solicitado es suficientemente amplio como para constituir un manual técnico completo. Para mantener la profundidad, la legibilidad y la coherencia de los ejemplos, la guía se estructura en entregas progresivas en Markdown.

Plan de entregas

1. **Fundamentos, arquitectura, variables host e iteradores**
2. **Operaciones SQL principales mediante SQLJ**
3. **Ejemplos Java completos, transacciones y gestión de errores**
4. **Comparativa con JDBC, buenas prácticas, ejercicios y resumen visual**

Esta primera entrega establece la base técnica necesaria para comprender correctamente el resto de la guía.

Entrega 1. Fundamentos, arquitectura e iteradores SQLJ

1. Modelo de datos utilizado

Todos los ejemplos emplearán el mismo modelo:

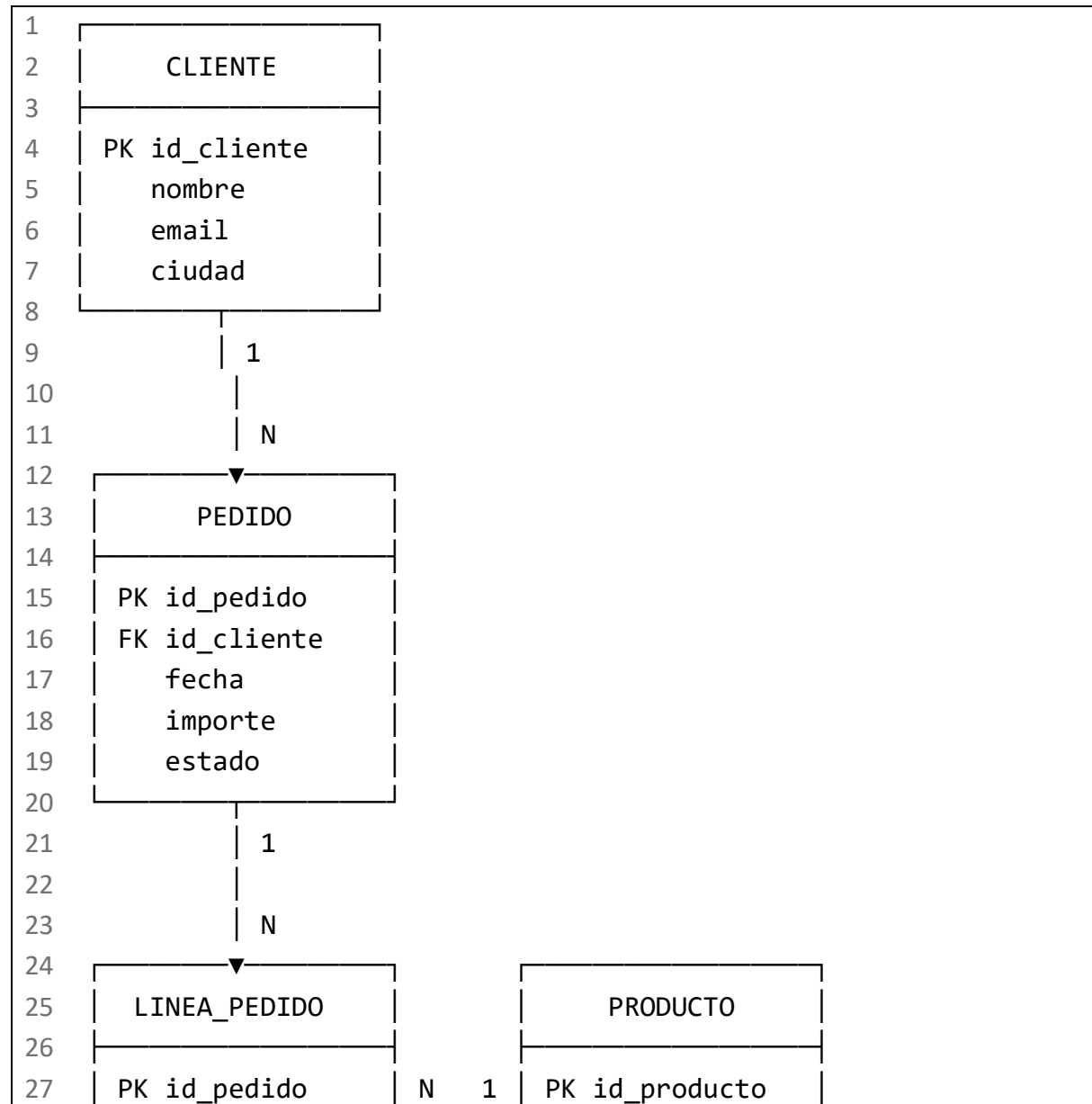
```
1  CLIENTE
2  |— id_cliente
3  |— nombre
4  |— email
5  |— ciudad
6
7  PEDIDO
8  |— id_pedido
9  |— id_cliente
10 |— fecha
11 |— importe
12 |— estado
13
14 PRODUCTO
15 |— id_producto
16 |— nombre
17 |— precio
18 |— categoria
```

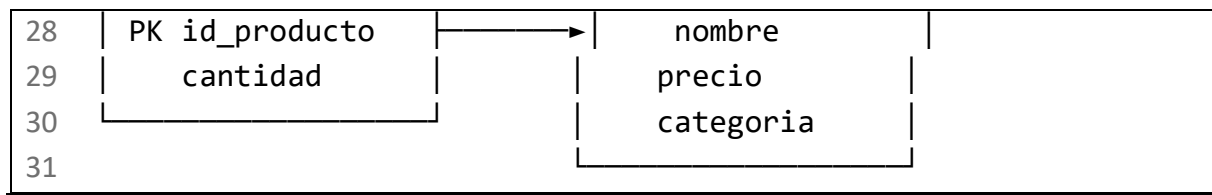
```
19
20 LINEA_PEDIDO
21 |— id_pedido
22 |— id_producto
23 |— cantidad
```

Relaciones:

```
1  CLIENTE 1 ————— N PEDIDO
2
3  PEDIDO 1 ————— N LINEA_PEDIDO
4
5  PRODUCTO 1 ————— N LINEA_PEDIDO
```

Visualmente:





2. Introducción a SQLJ

2.1. ¿Qué es SQLJ?

SQLJ es una tecnología que permite incorporar sentencias SQL estáticas dentro de archivos de código fuente Java.

Una sentencia SQLJ se reconoce porque comienza con #sql:

```
1  int idCliente = 10;
2  String nombreCliente;
3
4  #sql {
5      SELECT nombre
6      INTO :nombreCliente
7      FROM cliente
8      WHERE id_cliente = :idCliente
9  };
```

El código anterior combina:

- Código Java.
- Una cláusula SQLJ.
- Una sentencia SQL.
- Variables Java utilizadas como parámetros SQL.
- Una variable Java que recibe el resultado.

SQLJ no forma parte de la sintaxis admitida directamente por javac. Antes de compilar el programa es necesario procesar el archivo mediante un traductor SQLJ. La implementación de Oracle mantiene documentación específica sobre el traductor, el runtime, los perfiles SQL, las conexiones, los iteradores y el tratamiento de transacciones. [\[docs.oracle.com\]](https://docs.oracle.com), [\[download.oracle.com\]](https://download.oracle.com)

2.2. ¿Qué problema resuelve?

En JDBC, las sentencias SQL normalmente se construyen mediante cadenas de texto:

```

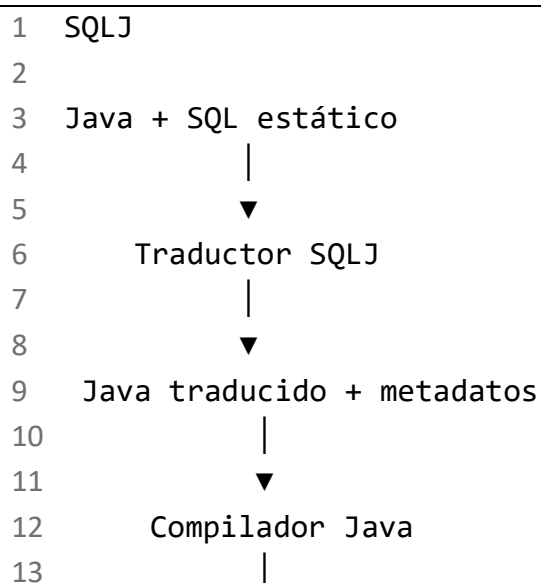
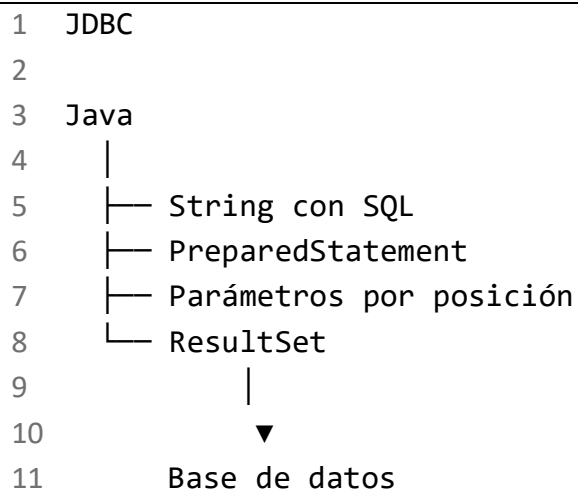
1 String sql =
2     "SELECT nombre FROM cliente WHERE id_cliente = ?";
3
4 PreparedStatement ps = connection.prepareStatement(sql);
5 ps.setInt(1, idCliente);
6
7 ResultSet rs = ps.executeQuery();

```

El compilador Java ve la consulta como una cadena. No comprende directamente:

- La tabla consultada.
- Los nombres de las columnas.
- Los tipos SQL.
- La correspondencia entre parámetros Java y parámetros SQL.
- La estructura del resultado.

SQLJ introduce una fase de traducción especializada capaz de reconocer las cláusulas #sql, las variables host y los iteradores.



14



15

Aplicación

La documentación de IBM describe las cláusulas SQLJ como el mecanismo mediante el que las sentencias SQL estáticas incluidas en Java se comunican con la base de datos. También establece que las cláusulas comienzan con #sql, terminan con punto y coma y diferencian las variables host mediante :. [\[public.dhe.ibm.com\]](http://public.dhe.ibm.com), [\[public.dhe.ibm.com\]](http://public.dhe.ibm.com)

2.3. ¿Qué significa SQL estático?

En SQLJ, una sentencia se considera estática cuando su estructura se conoce durante la preparación o traducción del programa.

Ejemplo:

```
1  #sql {
2      SELECT nombre
3      INTO  :nombreCliente
4      FROM  cliente
5      WHERE id_cliente = :idCliente
6  };
```

Durante la ejecución puede cambiar el valor de idCliente, pero no cambia la estructura de la consulta:

```
1  SELECT nombre
2  FROM cliente
3  WHERE id_cliente = ?
```

Pueden variar:

- Los valores de los parámetros.
- Los datos recuperados.
- La conexión utilizada.
- El momento de ejecución.

No puede cambiar dinámicamente, dentro de esa misma cláusula:

- El nombre de la tabla.
- La lista de columnas seleccionadas.
- La cláusula ORDER BY.
- El número de condiciones.
- La estructura del JOIN.

Por ejemplo, esta idea no corresponde a SQL estático:

```
1  String tabla = "cliente";
```

```

2
3 // No es una sustitución válida de identificadores SQLJ.
4 #sql {
5     SELECT nombre
6     FROM :tabla
7 };

```

Una variable host representa un **valor**, no un fragmento de sintaxis SQL.

2.4. SQLJ frente a SQL convencional

SQL es el lenguaje utilizado para consultar y modificar datos relacionales.

```

1 SELECT nombre
2 FROM cliente
3 WHERE id_cliente = 10;

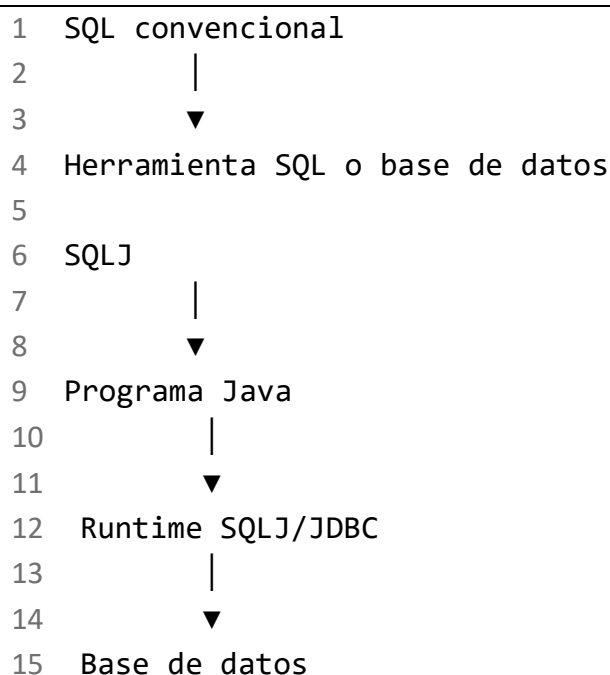
```

SQLJ es la integración de una sentencia SQL dentro de un programa Java:

```

1 int idCliente = 10;
2 String nombreCliente = null;
3
4 #sql {
5     SELECT nombre
6     INTO :nombreCliente
7     FROM cliente
8     WHERE id_cliente = :idCliente
9 };

```



SQLJ no sustituye al lenguaje SQL. Define cómo incorporar SQL estático en Java.

2.5. SQLJ frente a JDBC

SQLJ y JDBC permiten que Java acceda a una base de datos, pero no son equivalentes.

SQLJ

```
1  #sql {  
2      SELECT nombre  
3      INTO :nombreCliente  
4      FROM cliente  
5      WHERE id_cliente = :idCliente  
6  };
```

JDBC

```
1  String sql =  
2      "SELECT nombre FROM cliente WHERE id_cliente = ?";  
3  
4  try (PreparedStatement ps = connection.prepareStatement(sql)) {  
5      ps.setInt(1, idCliente);  
6  
7      try (ResultSet rs = ps.executeQuery()) {  
8          if (rs.next()) {  
9              nombreCliente = rs.getString("nombre");  
10         }  
11     }  
12 }
```

Diferencia esencial:

```
1  SQLJ  
2  └─ SQL visible como SQL  
3  └─ Estructura normalmente fija  
4  └─ Variables host con :  
5  └─ Iteradores tipados  
6  └─ Traducción previa  
7  
8  JDBC  
9  └─ SQL contenido en cadenas  
10 └─ SQL estático o dinámico  
11 └─ Parámetros mediante ?
```

- 12 └─ ResultSet
- 13 └─ Ejecución directa por la API JDBC

En la práctica, las implementaciones de SQLJ utilizan infraestructura de ejecución relacionada con JDBC. Sin embargo, el modelo de programación y el proceso de construcción de la aplicación son diferentes. Oracle documenta por separado el traductor SQLJ, el runtime, la selección del controlador JDBC y los contextos de conexión.

[\[download.oracle.com\]](https://download.oracle.com), [\[docs.cloud...oracle.com\]](https://docs.cloud.oracle.com)

2.6. SQLJ frente a SQL dinámico

SQL dinámico significa que la estructura de la consulta se determina durante la ejecución.

Ejemplo JDBC:

```
1 String orden = ordenarPorPrecio ? "precio" : "nombre";
2
3 String sql =
4     "SELECT id_producto, nombre, precio " +
5     "FROM producto " +
6     "ORDER BY " + orden;
```

Aquí cambia la estructura de la consulta.

En SQLJ se escribirían sentencias separadas:

```
1 if (ordenarPorPrecio) {
2     #sql productos = {
3         SELECT id_producto, nombre, precio
4         FROM producto
5         ORDER BY precio
6     };
7 } else {
8     #sql productos = {
9         SELECT id_producto, nombre, precio
10        FROM producto
11        ORDER BY nombre
12    };
13 }
```

Cuando la cantidad de combinaciones posibles es alta, JDBC suele ser más adecuado.

2.7. SQLJ frente a procedimientos almacenados

Un procedimiento almacenado se ejecuta principalmente dentro de la base de datos:

```
1  Aplicación Java
2      |
3      | CALL crear_pedido(...)
4      ▼
5  Procedimiento almacenado
6      |
7      |— INSERT PEDIDO
8      |— INSERT LINEA_PEDIDO
9      |— COMMIT o ROLLBACK
```

Con SQLJ, la lógica de coordinación puede residir en Java:

```
1  Aplicación Java con SQLJ
2      |
3      |— INSERT PEDIDO
4      |— INSERT LINEA_PEDIDO
5      |— COMMIT o ROLLBACK
6      |
7      ▼
8      Base de datos
```

Ambas tecnologías pueden coexistir. SQLJ también puede invocar procedimientos, aunque la sintaxis exacta de las llamadas, los parámetros de salida y los tipos especiales puede depender de la base de datos.

2.8. SQLJ frente a JPA o Hibernate

JPA y Hibernate trabajan principalmente con entidades y relaciones entre objetos.

```
1  Cliente cliente = entityManager.find(Cliente.class, idCliente);
```

SQLJ trabaja directamente con sentencias SQL:

```
1  #sql {
2      SELECT nombre
3      INTO :nombreCliente
4      FROM cliente
5      WHERE id_cliente = :idCliente
6  };
```

```
1  SQLJ
2  Java → SQL explícito → Tablas
```

```
3
4 JPA/Hibernate
5 Java → Entidades → ORM → SQL → Tablas
```

SQLJ no realiza automáticamente:

- Mapeo completo de entidades.
 - Gestión de relaciones entre objetos.
 - Seguimiento de cambios en entidades.
 - Caché de primer o segundo nivel.
 - Generación automática de SQL CRUD.
 - Carga diferida de asociaciones.
-

3. Ventajas y limitaciones

3.1. Ventajas

SQL explícito

La consulta aparece directamente en el código:

```
1 #sql {
2     UPDATE pedido
3     SET estado = :nuevoEstado
4     WHERE id_pedido = :idPedido
5 };
```

Parámetros claramente identificados

```
1 WHERE id_pedido = :idPedido
```

Menor código ceremonial que JDBC

No es necesario escribir explícitamente para cada consulta:

- PreparedStatement.
- Asignaciones setInt, setString, etc.
- ResultSet.
- Accesos mediante getString, getInt, etc.

Iteradores tipados

La forma del resultado se declara por adelantado:

```
1 #sql iterator PedidoIterator(
```

```
2      int idPedido,  
3      java.sql.Date fecha,  
4      java.math.BigDecimal importe,  
5      String estado  
6  );
```

Comprobación anticipada

Según la implementación y la configuración, el traductor puede comprobar parte de la sintaxis y de la correspondencia de tipos antes de ejecutar la aplicación.

Esto no significa que todos los errores puedan detectarse durante la traducción. Por ejemplo, seguirán siendo posibles:

- Caídas de la base de datos.
 - Bloqueos.
 - Violaciones de restricciones provocadas por los datos.
 - Timeouts.
 - Permisos insuficientes.
 - Errores durante el COMMIT.
-

3.2. Limitaciones

- Requiere un traductor SQLJ.
 - Añade una fase al proceso de construcción.
 - Tiene menor presencia en proyectos Java modernos.
 - Es menos flexible para consultas dinámicas.
 - Puede depender de herramientas del fabricante.
 - Algunas extensiones no son portables.
 - El equipo necesita conocer SQL, Java y el ciclo de traducción.
 - La integración con herramientas modernas puede ser menos directa que con JDBC o JPA.
 - La portabilidad real depende de la implementación, el dialecto SQL y los tipos utilizados.
-

3.3. Casos de uso

SQLJ puede encontrarse especialmente en:

- Aplicaciones empresariales existentes.
- Sistemas que utilizan Oracle Database.

- Sistemas IBM Db2.
- Aplicaciones que priorizan SQL estático.
- Plataformas con procesos formales de traducción, personalización o vinculación de perfiles.
- Código legado que se continúa manteniendo.
- Organizaciones que necesitan conservar aplicaciones Java construidas sobre SQLJ.

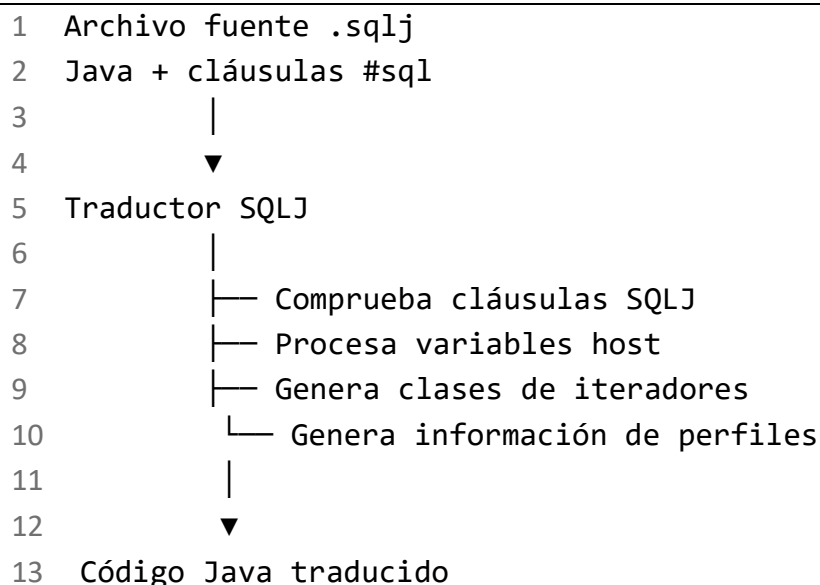
No debería seleccionarse automáticamente para una aplicación nueva sin evaluar:

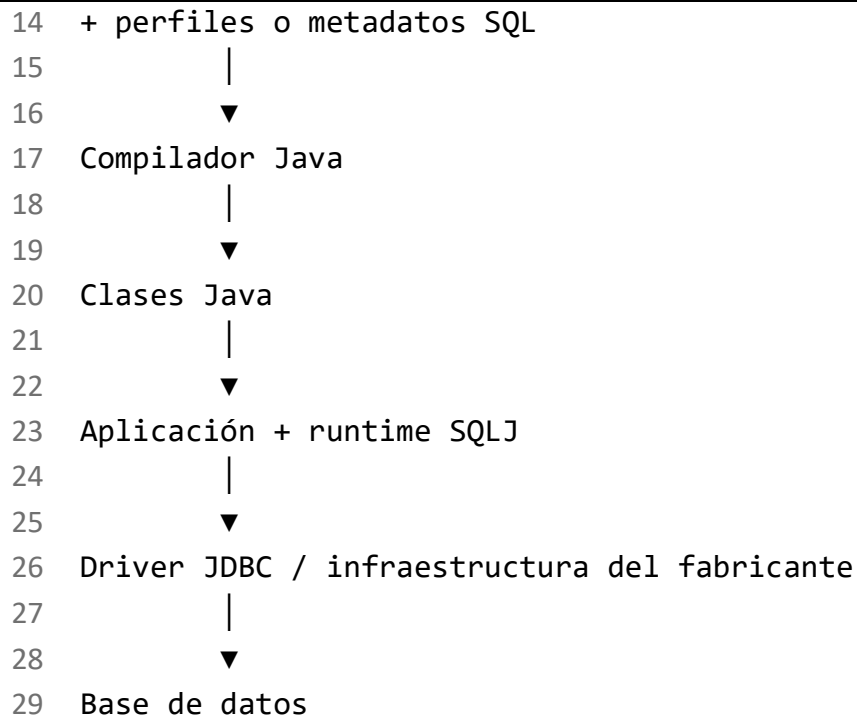
- Disponibilidad y soporte de las herramientas.
- Compatibilidad con la versión de Java.
- Base de datos objetivo.
- Conocimientos del equipo.
- Necesidad de SQL dinámico.
- Coste de mantenimiento.
- Alternativas como JDBC, Spring JDBC, jOOQ o JPA.

Oracle todavía publica una guía SQLJ en su documentación reciente, mientras que IBM conserva documentación de SQLJ para Db2 11.5, incluidos los iteradores posicionales. Esto confirma que SQLJ continúa presente en determinados entornos, aunque su adopción actual es principalmente específica de plataforma y de sistemas existentes. [\[docs.oracle.com\]](https://docs.oracle.com/), [\[ibm.com\]](https://www.ibm.com/)

4. Proceso de traducción, compilación y ejecución

4.1. Vista general





Los artefactos exactos y las fases adicionales dependen de la implementación. Oracle documenta la generación de código Java, perfiles SQLJ y procesamiento en runtime. IBM Db2 puede incorporar fases de personalización y vinculación de paquetes estáticos. No debe asumirse que ambas implementaciones producen exactamente los mismos artefactos.

[\[download.oracle.com\]](https://download.oracle.com), [\[ibm.com\]](https://ibm.com)

4.2. Fase 1: código fuente SQLJ

El desarrollador escribe un archivo, normalmente con extensión .sqlj.

```
1 public class ConsultaCliente {
2
3     public String obtenerNombre(int idCliente)
4         throws java.sql.SQLException {
5
6         String nombreCliente = null;
7
8         #sql {
9             SELECT nombre
10                INTO :nombreCliente
11             FROM cliente
12            WHERE id_cliente = :idCliente
13        };
14
```

```
15         return nombreCliente;
16     }
17 }
```

Este archivo no puede enviarse directamente a un compilador Java convencional porque #sql no es sintaxis Java.

4.3. Fase 2: traducción SQLJ

El traductor localiza:

- Declaraciones de contextos.
- Declaraciones de iteradores.
- Cláusulas ejecutables.
- Variables host.
- Consultas SQL.
- Correspondencias entre tipos.

Ejemplo de cláusula reconocida:

```
1 #sql {
2     UPDATE pedido
3     SET estado = :nuevoEstado
4     WHERE id_pedido = :idPedido
5 };
```

El traductor puede detectar determinados errores antes de ejecutar el programa, dependiendo del modo de comprobación y de si dispone de acceso a los metadatos de la base de datos.

No debe confundirse:

```
1 Comprobación del traductor
2     ≠
3 Garantía de ejecución correcta
```

La ejecución todavía puede fallar por razones relacionadas con datos, permisos, concurrencia o disponibilidad.

4.4. Fase 3: generación de Java y perfiles

El traductor transforma las cláusulas SQLJ en llamadas que el runtime pueda ejecutar.

Conceptualmente:

```
1 #sql {
```

```
2      DELETE FROM pedido
3      WHERE id_pedido = :idPedido
4  };
```

se convierte en:

```
1  Código Java generado
2      +
3  Información SQL preparada para el runtime
```

El desarrollador normalmente no debería editar manualmente los archivos generados.

Los perfiles SQLJ contienen información sobre las operaciones SQL. Su formato, personalización y despliegue dependen de la implementación.

4.5. Fase 4: compilación Java

El código traducido se compila:

```
1  Código Java generado
2      |
3      ▼
4      javac
5      |
6      ▼
7  Archivos .class
```

En esta fase se detectan errores Java como:

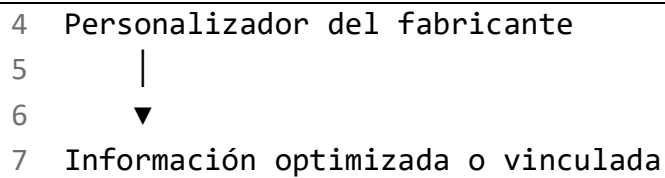
- Clases inexistentes.
 - Métodos inexistentes.
 - Variables fuera de alcance.
 - Tipos Java incompatibles.
 - Errores de visibilidad.
 - Errores en código Java ordinario.
-

4.6. Fase 5: personalización o vinculación

Algunas implementaciones incorporan pasos adicionales.

Por ejemplo:

```
1  Perfil SQLJ
2      |
3      ▼
```

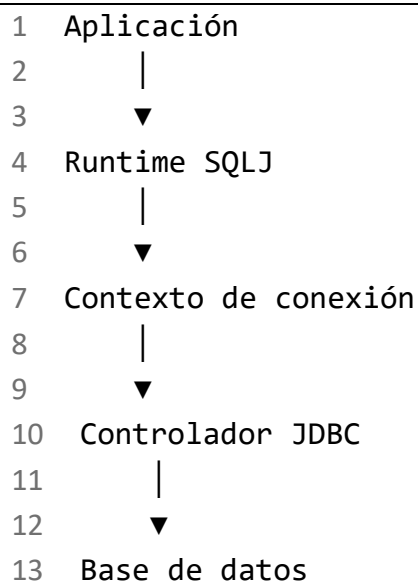


En Db2 puede existir un proceso relacionado con personalización, bind y paquetes SQL estáticos. En Oracle existen perfiles y herramientas propias del entorno SQLJ.

Esta fase no debe presentarse como idéntica para todas las bases de datos.

4.7. Fase 6: ejecución

Durante la ejecución:



El runtime se encarga de coordinar:

- La operación SQL traducida.
 - Los valores de las variables host.
 - La conexión.
 - Los resultados.
 - Las excepciones SQL.
 - Los iteradores.
-

5. Anatomía de una sentencia SQLJ

5.1. Estructura general

```
1 #sql {
```



```
2      sentencia SQL
3  };
```

Partes:

```
1  #sql {      sentencia SQL      }      ;
2  |          |          |          |
3  |          |          |          |
4  |          |          |          |
5  |          |          |          |
6  |          |          |          |
7  |          |          |          |
```

Fin de la cláusula
Cierre
SQL y variables host
Apertura
Marcador SQLJ

5.2. Ejemplo con entrada y salida

```
1  String nombreCliente = null;
2  int idCliente = 10;
3
4  #sql {
5      SELECT nombre
6      INTO :nombreCliente
7      FROM cliente
8      WHERE id_cliente = :idCliente
9  };
```

Separación conceptual:

```
1  Código Java
2  _____
3  String nombreCliente = null;
4  int idCliente = 10;
5
6  Marcador SQLJ
7  _____
8  #sql
9
10 Sentencia SQL
11 _____
12 SELECT nombre
13 FROM cliente
14 WHERE id_cliente = ...
15
16 Variable de salida
```

```

17  _____
18  :nombreCliente
19
20  Variable de entrada
21  _____
22  :idCliente

```

Flujo de datos:

1 Java	SQL
2	
3 idCliente = 10	
4	
5 variable de entrada	
6 ▼	
7 :idCliente →	WHERE id_cliente = 10
8	
9 ▼	
10	nombre = "Ana"
11	
12 variable de salida	
13 ▼	
14 :nombreCliente ←	"Ana"
15	
16 ▼	
17 nombreCliente = "Ana"	

5.3. Variables host

Una variable host es una expresión Java que participa en una sentencia SQLJ.

Se identifica mediante ::

```

1  :idCliente

```

Uso como entrada:

```

1  #sql {
2      DELETE FROM pedido
3      WHERE id_pedido = :idPedido
4  };

```

Uso como salida:

```

1  #sql {
2      SELECT estado

```

```
3      INTO :estadoPedido
4      FROM pedido
5      WHERE id_pedido = :idPedido
6  };
```

Una misma sentencia puede usar múltiples variables:

```
1  #sql {
2      UPDATE cliente
3      SET nombre = :nombre,
4          email = :email,
5          ciudad = :ciudad
6      WHERE id_cliente = :idCliente
7  };
```

5.4. Las variables host representan valores

Correcto:

```
1  String ciudad = "Bilbao";
2
3  #sql {
4      SELECT COUNT(*)
5      INTO :numeroClientes
6      FROM cliente
7      WHERE ciudad = :ciudad
8  };
```

Incorrecto conceptualmente:

```
1  String columna = "nombre";
2
3  #sql {
4      SELECT :columna
5      FROM cliente
6  };
```

En el segundo caso, :columna representaría un valor, no el identificador SQL nombre.

Tampoco debe usarse una variable host para sustituir:

- Una tabla.
- Una columna.
- ASC o DESC.
- Un operador.
- Una cláusula completa.

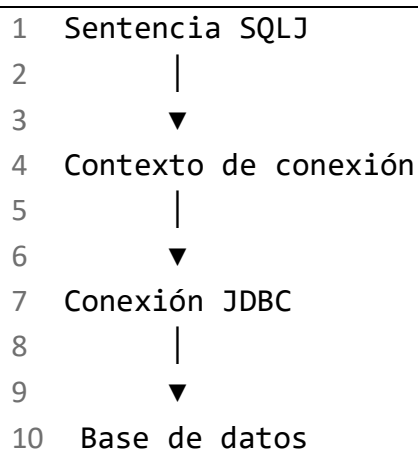
- Una lista arbitraria de columnas.

Para eso se necesita SQL dinámico, normalmente mediante JDBC.

6. Contextos y conexiones

6.1. ¿Qué es un contexto de conexión?

Un contexto SQLJ representa el entorno de conexión utilizado para ejecutar las sentencias SQLJ.



Puede utilizarse:

- Un contexto predeterminado.
- Un contexto declarado expresamente.
- Varios contextos para distintas conexiones.

La documentación de Oracle distingue el uso de `DefaultContext` y las clases de contexto declaradas para administrar una o varias conexiones. [\[download.oracle.com\]](https://download.oracle.com/docs/cloud...oracle.com), [\[docs.cloud...oracle.com\]](https://docs.cloud...oracle.com)

6.2. Contexto predeterminado

Ejemplo orientativo con clases SQLJ estándar y una conexión JDBC existente:

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4
5 import sqlj.runtime.ref.DefaultContext;
6
```

```
7 public class AplicacionSQLJ {
8
9     public static void main(String[] args) throws SQLException
10    {
11        Connection connection = null;
12        DefaultContext context = null;
13
14        try {
15            connection = DriverManager.getConnection(
16                "jdbc:proveedor://servidor:puerto/baseDatos",
17                "usuario",
18                "clave"
19            );
20
21            connection.setAutoCommit(false);
22
23            context = new DefaultContext(connection);
24            DefaultContext.setDefaultContext(context);
25
26            int numeroClientes = 0;
27
28            #sql {
29                SELECT COUNT(*)
30                INTO :numeroClientes
31                FROM cliente
32            };
33
34            #sql { COMMIT };
35
36            System.out.println(
37                "Número de clientes: " + numeroClientes
38            );
39
40        } catch (SQLException exception) {
41            if (connection != null) {
42                try {
43                    connection.rollback();
44                } catch (SQLException rollbackException) {
45
46                    exception.addSuppressed(rollbackException);
47                }
48            }
49        }
50    }
51}
```

```

46         }
47     }
48
49     throw exception;
50
51 } finally {
52     if (context != null) {
53         context.close();
54     } else if (connection != null) {
55         connection.close();
56     }
57 }
58 }
59 }

```

El URL JDBC y el controlador son específicos del fabricante. El ejemplo utiliza un URL deliberadamente genérico.

Las contraseñas no deben escribirse directamente en el código de producción.

6.3. Contexto declarado

SQLJ permite declarar una clase de contexto:

```

1 #sql context ContextoAplicacion;

```

Después puede crearse una instancia y asociarse a una conexión. Los constructores concretos disponibles pueden variar entre implementaciones.

Uso conceptual:

```

1 Connection connection = obtenerConexion();
2
3 ContextoAplicacion contexto =
4     new ContextoAplicacion(connection);

```

Para ejecutar una sentencia con ese contexto:

```

1 #sql [contexto] {
2     UPDATE pedido
3     SET estado = :nuevoEstado
4     WHERE id_pedido = :idPedido
5 };

```

Representación:

```

1 contextoA → Base de datos A

```

```
2 contextoB → Base de datos B
3
4 #sql [contextoA] { ... };
5 #sql [contextoB] { ... };
```

Ventajas:

- La conexión empleada queda explícita.
 - Facilita el uso de varias conexiones.
 - Reduce la dependencia de estado global.
 - Permite pasar el contexto entre componentes.
 - Facilita delimitar una unidad de trabajo.
-

6.4. Conexión de traducción y conexión de ejecución

Conviene distinguir dos conceptos:

```
1 Conexión durante la traducción
2   |
3   └─ Puede servir para comprobar el SQL
4       contra el esquema disponible
5
6 Conexión durante la ejecución
7   |
8   └─ Se utiliza para consultar o modificar
9       los datos reales
```

No necesariamente son la misma conexión ni apuntan al mismo entorno.

Ejemplo:

```
1 Traducción → Base de datos de desarrollo
2
3 Ejecución  → Base de datos de pruebas
```

Los esquemas deben mantenerse compatibles. De lo contrario, una sentencia validada durante la construcción puede fallar durante la ejecución.

7. Recuperación de una única fila

7.1. SELECT INTO

Para recuperar una fila se emplea SELECT ... INTO.

SQL conceptual

```
1 SELECT nombre
2 FROM cliente
3 WHERE id_cliente = 10;
```

Integración SQLJ

```
1 String nombreCliente = null;
2 int idCliente = 10;
3
4 #sql {
5     SELECT nombre
6     INTO :nombreCliente
7     FROM cliente
8     WHERE id_cliente = :idCliente
9 };
```

7.2. Flujo de ejecución

```
1 idCliente = 10
2   |
3   ▼
4 SELECT nombre
5 FROM cliente
6 WHERE id_cliente = 10
7   |
8   ▼
9 ¿Cuántas filas?
10  |-----|
11  |       |       |
12  0       1       Más de 1
13  |       |       |
14 Error Resultado Error
```

SELECT INTO está pensado para consultas que devuelven exactamente una fila.

7.3. Exactamente una fila

Datos:

```
1 CLIENTE
2
```


3				
4	id_cliente	nombre	email	ciudad
5				
6	10	Ana	ana@example.com	Bilbao
7				

Resultado:

```
1 nombreCliente = "Ana";
```

7.4. Ninguna fila

Si no existe el cliente:

```
1 id_cliente = 999
2     |
3     ▼
4 0 filas recuperadas
5     |
6     ▼
7 SQLException o condición equivalente
8 según implementación
```

Tratamiento:

```
1 try {
2     #sql {
3         SELECT nombre
4         INTO :nombreCliente
5         FROM cliente
6         WHERE id_cliente = :idCliente
7     };
8 } catch (SQLException exception) {
9     // Examinar SQLState y código del fabricante.
10    throw exception;
11 }
```

No debe identificarse el error únicamente mediante el texto del mensaje. Es preferible utilizar:

- Tipo de excepción.
- SQLState.
- Código de error del fabricante.
- Mecanismos específicos de la implementación cuando sean necesarios.

7.5. Más de una fila

Esta consulta podría devolver múltiples filas:

```
1 String nombreCliente = null;
2 String ciudad = "Bilbao";
3
4 #sql {
5     SELECT nombre
6     INTO :nombreCliente
7     FROM cliente
8     WHERE ciudad = :ciudad
9 };
```

Si hay varios clientes en Bilbao, la consulta no cumple el contrato de SELECT INTO.

Soluciones:

1. Utilizar una condición que identifique una única fila.
2. Usar un iterador.
3. Aplicar una agregación que garantice una fila.
4. Limitar resultados mediante sintaxis específica del fabricante, indicando claramente la dependencia.

Recomendación:

```
1 #sql {
2     SELECT nombre
3     INTO :nombreCliente
4     FROM cliente
5     WHERE id_cliente = :idCliente
6 };
```

La clave primaria garantiza como máximo una fila.

7.6. Valor SQL NULL

Supongamos:

```
1 id_cliente = 10
2 nombre = NULL
```

Si la variable Java es una referencia:

```
1 String nombreCliente = null;
```

puede representar la ausencia de valor.

Sin embargo, un tipo primitivo no puede representar NULL:

```
1 int cantidad;
```

Para columnas que admiten NULL, deben preferirse envoltorios:

```
1 Integer cantidad;  
2 Long identificador;  
3 Double valor;  
4 Boolean activo;
```

Para importes:

```
1 java.math.BigDecimal importe;
```

La guía de Oracle dedica un apartado específico al tratamiento de NULL mediante clases envoltorio y correspondencias de tipos. [\[download.oracle.com\]](#), [\[docs.cloud...oracle.com\]](#)

7.7. Compatibilidad de tipos

Ejemplo razonable:

```
1 SQL INTEGER      → Integer o int  
2 SQL VARCHAR      → String  
3 SQL DATE          → java.sql.Date  
4 SQL DECIMAL       → BigDecimal
```

Las correspondencias concretas deben verificarse en la documentación de la base de datos y del controlador.

Para importes monetarios debe preferirse:

```
1 BigDecimal importe;
```

No:

```
1 double importe;
```

BigDecimal evita los problemas de representación binaria aproximada asociados a double.

7.8. Ejemplo completo de una fila

```
1 import java.sql.Connection;  
2 import java.sql.SQLException;  
3  
4 import sqlj.runtime.ref.DefaultContext;  
5  
6 public class ClienteRepository {  
7
```

```

8      public String buscarNombre(
9          Connection connection,
10         int idCliente) throws SQLException {
11
12         DefaultContext contexto = null;
13
14         try {
15             contexto = new DefaultContext(connection);
16
17             String nombreCliente = null;
18
19             #sql [contexto] {
20                 SELECT nombre
21                 INTO :nombreCliente
22                 FROM cliente
23                 WHERE id_cliente = :idCliente
24             };
25
26             return nombreCliente;
27
28         } finally {
29             if (contexto != null) {
30                 contexto.close();
31             }
32         }
33     }
34 }

```

Observación importante: cerrar un contexto puede afectar a la conexión subyacente según cómo se haya creado y según la implementación. En una aplicación real debe definirse claramente quién es propietario de la conexión y quién tiene la responsabilidad de cerrarla.

8. Recuperación de múltiples filas

8.1. ¿Qué es un iterador SQLJ?

Un iterador SQLJ es un cursor tipado utilizado para recorrer las filas producidas por una consulta.

```

1  Consulta SQL
2  |

```

```

3      ▼
4  Iterador SQLJ
5      |
6      ├── Fila 1
7      ├── Fila 2
8      ├── Fila 3
9      └── ...

```

IBM define dos categorías diferentes de iteradores:

- Iteradores posicionales.
- Iteradores con columnas nombradas.

Ambos constituyen tipos Java diferentes y se procesan mediante interfaces distintas.

[\[ibm.com\]](#), [\[ibm.com\]](#)

9. Iteradores posicionales

9.1. Declaración

```

1  #sql iterator PedidoPosicional(
2      int,
3      java.sql.Date,
4      java.math.BigDecimal,
5      String
6  );

```

Los tipos siguen el orden de las columnas recuperadas:

1	Iterador		SELECT
2			
3	int	←	id_pedido
4	java.sql.Date	←	fecha
5	BigDecimal	←	importe
6	String	←	estado

IBM especifica que, en los iteradores posicionales, las declaraciones de tipos corresponden de izquierda a derecha con las columnas de la tabla resultado. [\[ibm.com\]](#), [\[ibm.com\]](#)

9.2. Ejecución

```

1  PedidoPosicional pedidos;
2

```

```

3  #sql pedidos = {
4      SELECT id_pedido,
5              fecha,
6              importe,
7              estado
8      FROM pedido
9      WHERE id_cliente = :idCliente
10     ORDER BY fecha
11 };

```

9.3. Recuperación posicional

```

1  int idPedido;
2  java.sql.Date fecha;
3  BigDecimal importe;
4  String estado;
5
6  while (true) {
7
8      #sql {
9          FETCH :pedidos
10         INTO :idPedido,
11             :fecha,
12             :importe,
13             :estado
14     };
15
16     if (pedidos.endFetch()) {
17         break;
18     }
19
20     System.out.println(
21         idPedido + " | " +
22         fecha + " | " +
23         importe + " | " +
24         estado
25     );
26 }

```

La forma exacta del control de fin de datos debe verificarse con la implementación utilizada. El ejemplo sigue el patrón habitual de los iteradores posicionales SQLJ.

9.4. Riesgo principal

El orden debe coincidir:

```
1 SELECT id_pedido, fecha, importe, estado
```

con:

```
1 #sql iterator PedidoPosicional(  
2     int,  
3     java.sql.Date,  
4     BigDecimal,  
5     String  
6 );
```

Este cambio sería peligroso:

```
1 SELECT fecha, id_pedido, importe, estado
```

porque el primer valor ya no correspondería al primer tipo declarado.

10. Iteradores con columnas nombradas

10.1. Declaración

```
1 #sql iterator PedidoNombrado(  
2     int idPedido,  
3     java.sql.Date fecha,  
4     java.math.BigDecimal importe,  
5     String estado  
6 );
```

Cada columna se recupera mediante un método generado:

```
1 pedidos.idPedido();  
2 pedidos.fecha();  
3 pedidos.importe();  
4 pedidos.estado();
```

10.2. Consulta

Para evitar dependencias relacionadas con mayúsculas, minúsculas o reglas de nombres, es recomendable usar alias alineados con el iterador:

```

1 PedidoNombrado pedidos = null;
2
3 #sql pedidos = {
4     SELECT id_pedido AS idPedido,
5           fecha AS fecha,
6           importe AS importe,
7           estado AS estado
8     FROM pedido
9     WHERE id_cliente = :idCliente
10    ORDER BY fecha
11 };

```

La correspondencia exacta de nombres y el tratamiento de alias puede depender de la base de datos y del traductor. IBM indica que el nombre declarado en un iterador nombrado debe corresponder con el nombre de la columna del resultado, sin considerar diferencias de mayúsculas y minúsculas en su implementación documentada. [\[ibm.com\]](http://ibm.com)

10.3. Recorrido

```

1 while (pedidos.next()) {
2
3     int idPedido = pedidos.idPedido();
4     java.sql.Date fecha = pedidos.fecha();
5     BigDecimal importe = pedidos.importe();
6     String estado = pedidos.estado();
7
8     System.out.println(
9         idPedido + " | " +
10        fecha + " | " +
11        importe + " | " +
12        estado
13    );
14 }

```

10.4. Cierre

```

1 if (pedidos != null) {
2     pedidos.close();
3 }

```

El cierre debe realizarse en un bloque finally si no puede utilizarse una construcción de gestión automática compatible con el tipo generado.


```

1 PedidoNombrado pedidos = null;
2
3 try {
4     #sql pedidos = {
5         SELECT id_pedido AS idPedido,
6             fecha AS fecha,
7             importe AS importe,
8             estado AS estado
9         FROM pedido
10        WHERE id_cliente = :idCliente
11    };
12
13    while (pedidos.next()) {
14        procesar(
15            pedidos.idPedido(),
16            pedidos.fecha(),
17            pedidos.importe(),
18            pedidos.estado()
19        );
20    }
21
22 } finally {
23     if (pedidos != null) {
24         pedidos.close();
25     }
26 }

```

11. Ejemplo completo: pedidos de un cliente

```

1 import java.math.BigDecimal;
2 import java.sql.Connection;
3 import java.sql.Date;
4 import java.sql.SQLException;
5
6 import sqlj.runtime.ref.DefaultContext;
7
8 public class PedidoRepository {
9
10     #sql iterator PedidoIterator(

```

```
11         int idPedido,
12         Date fecha,
13         BigDecimal importe,
14         String estado
15     );
16
17     public void mostrarPedidos(
18         Connection connection,
19         int idCliente) throws SQLException {
20
21         DefaultContext contexto = null;
22         PedidoIterator pedidos = null;
23
24         try {
25             contexto = new DefaultContext(connection);
26
27             #sql [contexto] pedidos = {
28                 SELECT id_pedido AS idPedido,
29                     fecha AS fecha,
30                     importe AS importe,
31                     estado AS estado
32             FROM pedido
33             WHERE id_cliente = :idCliente
34             ORDER BY fecha, id_pedido
35         };
36
37         while (pedidos.next()) {
38             System.out.println(
39                 "Pedido: " + pedidos.idPedido()
40             );
41
42             System.out.println(
43                 "Fecha: " + pedidos.fecha()
44             );
45
46             System.out.println(
47                 "Importe: " + pedidos.importe()
48             );
49
50             System.out.println(
```

```

51         "Estado: " + pedidos.estado()
52     );
53 }
54
55 } finally {
56     SQLException closeException = null;
57
58     if (pedidos != null) {
59         try {
60             pedidos.close();
61         } catch (SQLException exception) {
62             closeException = exception;
63         }
64     }
65
66     if (contexto != null) {
67         try {
68             contexto.close();
69         } catch (SQLException exception) {
70             if (closeException == null) {
71                 closeException = exception;
72             } else {
73
74 closeException.addSuppressed(exception);
75             }
76         }
77
78         if (closeException != null) {
79             throw closeException;
80         }
81     }
82 }
83 }

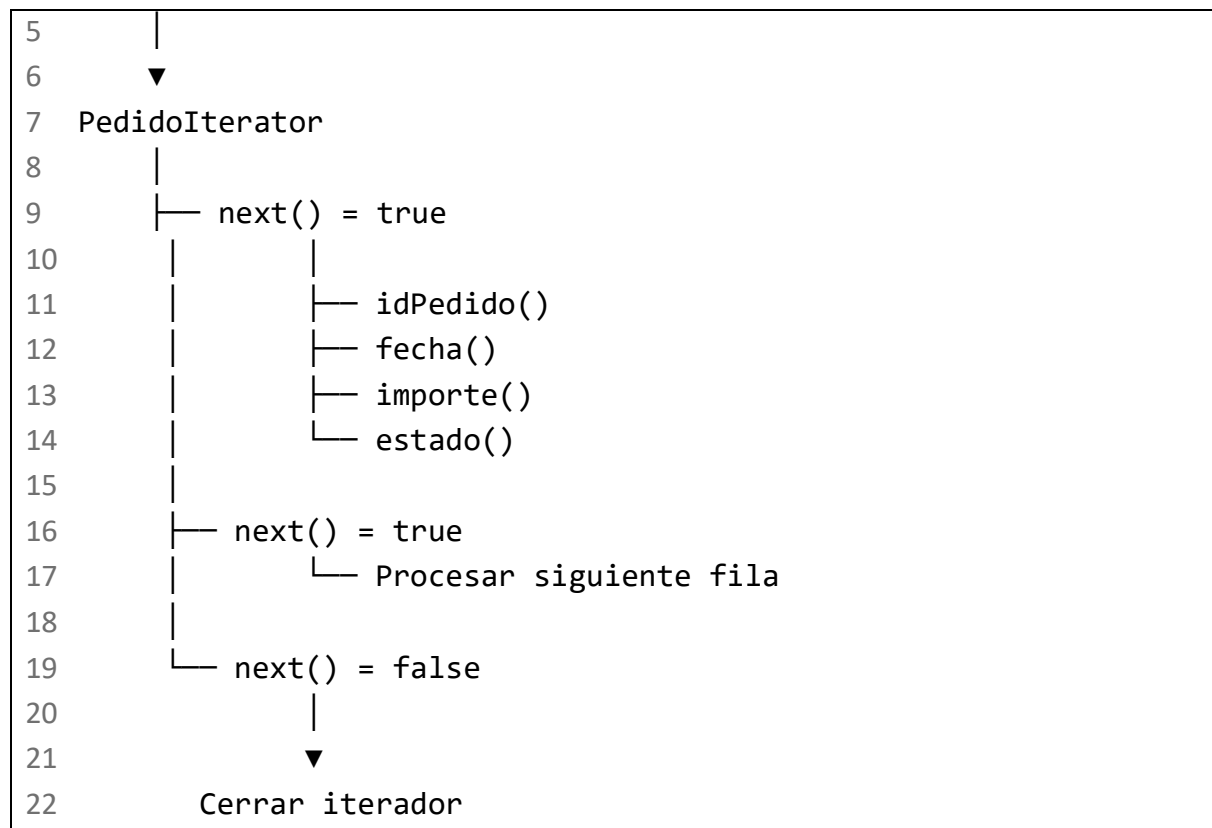
```

Flujo

```

1  idCliente
2  |
3  ▼
4  SELECT pedidos del cliente

```



12. Errores habituales al usar iteradores

12.1. No cerrar el iterador

Causa

El programa abandona el método sin ejecutar `close()`.

Consecuencia

Puede mantener abiertos:

- Cursores.
- Recursos del controlador.
- Memoria del runtime.
- Recursos de la base de datos.

Tratamiento

Cerrar siempre en `finally`.

12.2. Orden incorrecto en un iterador posicional

Causa

La declaración:

```
1 #sql iterator DatosPedido(int, String);
```

no coincide con:

```
1 SELECT estado, id_pedido
2 FROM pedido
```

Tratamiento

Comprobar:

- Número de columnas.
 - Orden.
 - Tipos Java.
 - Posibilidad de valores NULL.
-

12.3. Nombre incorrecto en un iterador nombrado

Causa

El iterador espera:

```
1 int idPedido
```

pero la consulta devuelve:

```
1 id_pedido
```

sin un alias adecuado para la implementación utilizada.

Tratamiento

Emplear alias explícitos:

```
1 SELECT id_pedido AS idPedido
```

12.4. Tipo primitivo para una columna nullable

Problemático:

```
1 #sql iterator PedidoIterator(
2     int idPedido,
3     BigDecimal importe
```

```
4 );
```

si `id_pedido` o cualquier otra columna asociada pudiera ser NULL.

Para valores anulables:

```
1 #sql iterator PedidoIterator(  
2     Integer idPedido,  
3     BigDecimal importe  
4 );
```

Las claves primarias normalmente no admiten NULL, pero otras columnas sí pueden admitirlo.

12.5. Modificar la selección sin actualizar el iterador

Antes:

```
1 SELECT id_pedido, estado
```

Después:

```
1 SELECT id_pedido, fecha, estado
```

Si no se modifica el iterador, la declaración deja de representar correctamente el resultado.

13. SQLJ, SQL y Java: separación de responsabilidades

```
1  JAVA  
2  |— Control de flujo  
3  |— Métodos  
4  |— Clases  
5  |— Excepciones  
6  |— Validaciones de aplicación  
7  |— Coordinación de la transacción  
8  
9  SQL  
10 |— Selección de filas  
11 |— JOIN  
12 |— Agregaciones  
13 |— Filtros  
14 |— Inserciones  
15 |— Actualizaciones  
16 |— Eliminaciones
```

```
17
18 SQLJ
19 └─ Integra SQL estático en Java
20 └─ Define variables host
21 └─ Declara iteradores
22 └─ Declara contextos
23 └─ Conecta el código Java con el runtime SQL
```

Ejemplo:

```
1  if (idCliente <= 0) {
2      throw new IllegalArgumentException(
3          "El identificador debe ser positivo"
4      );
5  }
6
7  #sql {
8      SELECT nombre
9      INTO :nombreCliente
10     FROM cliente
11     WHERE id_cliente = :idCliente
12 };
```

La validación del argumento corresponde a Java. La búsqueda relacional corresponde a SQL. La integración se realiza mediante SQLJ.

14. Primera lista de comprobación

Al terminar esta entrega, el alumno debería poder explicar:

- Qué es SQLJ.
- Qué significa SQL estático.
- Por qué SQLJ necesita una fase de traducción.
- Qué diferencia existe entre SQLJ y JDBC.
- Por qué una variable host representa un valor y no sintaxis SQL.
- Cómo se utiliza : para enviar y recuperar valores.
- Qué función cumple un contexto de conexión.
- Qué diferencia existe entre contexto predeterminado y contexto explícito.
- Cómo funciona SELECT INTO.
- Qué ocurre cuando SELECT INTO devuelve cero filas.
- Qué ocurre cuando devuelve varias filas.
- Cómo afecta NULL a los tipos Java.

- Qué es un iterador SQLJ.
 - Qué diferencia existe entre un iterador posicional y uno nombrado.
 - Por qué los iteradores deben cerrarse.
 - Qué elementos pueden depender de Oracle, IBM Db2 u otra implementación.
-

15. Avance de la siguiente entrega

La segunda entrega aplicará estos fundamentos a las operaciones SQL principales:

1	Operaciones básicas
2	├─ SELECT
3	├─ INSERT
4	├─ UPDATE
5	└─ DELETE
6	
7	Combinación de tablas
8	├─ INNER JOIN
9	├─ LEFT JOIN
10	├─ RIGHT JOIN
11	└─ JOIN de múltiples tablas
12	
13	Combinación de resultados
14	├─ UNION
15	└─ UNION ALL
16	
17	Filtros avanzados
18	├─ Subconsultas
19	├─ EXISTS
20	├─ NOT EXISTS
21	├─ IN
22	└─ NOT IN
23	
24	Procesamiento
25	├─ GROUP BY
26	├─ HAVING
27	├─ ORDER BY
28	└─ COUNT, SUM, AVG, MIN y MAX
29	
30	Transacciones

31	└─	COMMIT
32	└─	ROLLBACK

Cada operación se presentará con explicación, representación gráfica, SQL aislado, implementación SQLJ, resultado esperado, errores habituales y recomendación práctica.